Locating Sweep's 4×: Matched Work, a Faster Sub-Solver, and an Application Path That Searches Where Kissat Walks

Tactix development report for Andrew

4 September 2026

1 Objective and outcome

The objective is unchanged: bring Tactix within 10 % of Kissat's wall time on Tier D Ranks 1–40 while preserving N1–N4, with B28+C5 as the regression gate. Kissat's reference (`1930`) is 7,675.1 s, so the target is 8,442.7 s.

Outcome. Four commits were gated and pushed; `origin/main` is `2c30fd3`. Ranks 1–40 stand at **8,738.8 s**, which is **1.138**× Kissat, and B28+C5 is **0.491**. The headline moved little this session, and the reason matters more than the number: every one of the forty conflict counts is identical to the previous gate, so the change is behaviour-preserving and the ±14 s between the two totals is node-assignment noise rather than a measurement of the work landed.

The substantive result is diagnostic. Sweep is the largest single per-rule gap left on Tier D, and this session established exactly what that gap is made of. It is not search quality, not the embedded solver, and not any one hot line. It is that Tactix applies a discovered equivalence through a general substitution path that searches a list per clause, where Kissat walks one list per literal.

2 What the measurement says

Rank 22 spends 51.7 s in Sweep against Kissat's 12.9 s. The two do almost the same work.

	Tactix	Kissat
variables swept	221,816	196,771
encoded clauses	47.6 M	43.2 M
equivalences found	119,339	119,987
units found	1,072	1,118
sub-solver ticks	678 M	654 M

Every counter agrees within 10 %, and the ticks agree within 4 %. So this is a unit-cost gap at matched work, not a scheduling or dosing difference.

The embedded solver is not the cause either. In a paired source-line profile against a Kissat built with symbols, Tactix's sub-solver carries 1.30 % of samples where Kissat's `kitten` carries 2.40 %. Tactix's sub-solver is the faster of the two.

A call-graph profile then splits Sweep's 17.36 % inclusive share cleanly: finding the equivalences costs 2.0 %, and applying them costs 11.74 %. Roughly seven-eighths of Sweep is the write-back.

3 Why the write-back costs what it does

Kissat applies one equivalence with `substitute_connected_clauses`. That is a single compacting walk of the retired literal's dense list: each clause is rewritten in the arena in place, keeps its reference, and the moved watches are appended to the representative in one further pass. Nothing is searched.

Tactix builds a list of binary images, compacts the affected owners, pushes the images, replaces each large clause in place, and reconciles a parallel ledger that reproduces Kissat's single mixed watch stream on Tactix's separate indexes. Three steps of that path search a list once per clause: replacing a large clause searches the retired literal's occurrence vector, searches its ordered stream, and compacts both watch lists. Each search is linear in a degree that is itself the number of clauses already rewritten onto that representative, because a substituted class sends every retired literal's clauses to the same place. The cost is therefore quadratic in the class size.

4 What was repaired

Two instances of that pattern were removed, both verified by identical solver counters on interleaved same-node repeats.

	Sweep	Rank 22 total
baseline	52.2 s	292.7 s
incremental ledger republish	51.7 s	291.5 s
duplicate test grouped by owner	48.8 s	287.1 s

The ledger previously reconciled and republished every literal the whole Sweep call had ever touched, once per emission batch, which makes one environment pay for every earlier environment's footprint. It now carries the set of streams a batch actually mutated. A debug oracle re-derives the full publication on every batch and asserts the incremental one already matched it; it holds across the whole test suite.

The duplicate test was the clearer instance. Every rewritten binary asked whether its target pair was already live by scanning the target owner's binary prefix, from both endpoints. Grouping the images by owner and scanning each distinct owner once answers every image identically. A debug assertion re-derives the original per-image answer, and it caught a real error in the first attempt: the preceding compaction leaves a pair present in one endpoint's list and absent from the other, so both directions genuinely have to be asked.

5 Limitations and what was refuted

The tier headline did not move outside noise, and it would be wrong to present these repairs as having moved it. Rank 22 improved by 5.6 s in paired repeats; the tier total rose by 14.3 s with every conflict count unchanged. Both statements are true and the second is noise.

One further attempt measured neutral and was reverted rather than committed. Rank 21's elimination profile carries 2.9 % in `push_back` and 2.2 % in `free` where Kissat's carries neither, and BVE's occurrence audit constructed a literal vector for every clause it visits across 3.59 M candidate eliminations. Hoisting that vector and two others out of their loops kept every counter identical and changed the run by nothing measurable, so the allocator is not where that rank's elimination time goes.

Two documentation budgets are over: `PROGRESS.md` is 3,773 lines against 350, and `docs/implementation-notes.m` is 591 against 560. Compaction was not requested, so the debt is reported rather than paid.

6 Next concrete action

Give Sweep Kissat's connected walk in place of the general substitution path. The measurement above says what that is worth: about 36 s on Rank 22 alone, which is the largest single per-rule gap remaining on Tier D. The work is to replace the image-list-plus-ledger write-back with one compacting pass over the retired literal's stream, rewriting each clause in the arena and appending moved watches to the representative, so that no step is linear in the representative's growing degree.

Evidence for every number here is in `benchmarks/reports/2026-09-02-unit-cost-and-fidelity-stack.md`, sections 3.12 and 3.13, with campaigns `5010` through `5016`.